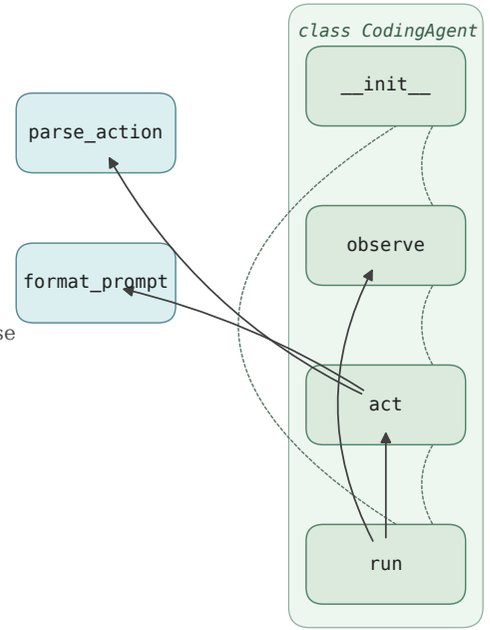
(a) Program dependency graph

```python
# example.py
def parse_action(t: str) -> Action:
    """Extract tool call."""
    m = ACTION_RE.search(t)
    if not m: return Action.noop()
    return Action(m['t'], m['a'])

def format_prompt(hist):
    return '\n'.join(map(str, hist))

class CodingAgent:
    def __init__(self, m): ...
    def observe(self, o):  ...
    def act(self, obs: str) -> Action:
        """Plan next action."""
        p = format_prompt(
            self.history + [obs])
        for _ in range(self.retries):
            t = self.model.generate(p)
            a = parse_action(t)
            if a.is_valid(): return a
        return Action.noop()
    def run(self, task) -> Traj:
        ...  # orchestrate observe/act
```

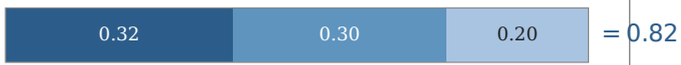
AST parse



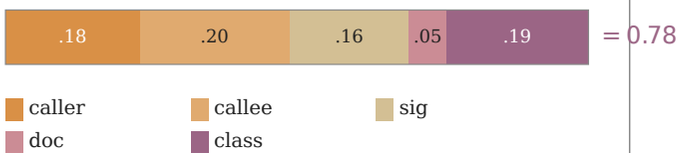
$\mathcal{E}_{call}$ $\mathcal{E}_{sib}$ class methods top-level fns

(b) $\hat{H}$ & $\hat{I}$ for `CodingAgent.act`

$\hat{H} = w_\ell\,\phi(\text{LoC}) + w_c\,\phi(\text{CC}) + w_d\,\phi(\text{D})$



$\hat{I} = \alpha C_{\text{caller}} + \beta C_{\text{callee}} + \gamma C_{\text{sig}} + \delta C_{\text{doc}} + \varepsilon C_{\text{class}}$



evidence: `run` calls `act` (caller); `act` calls `format_prompt`, `parse_action` typed signature + docstring; 3 in-class siblings via $\mathcal{E}_{sib}$.

(c) FIM$(v) = \dfrac{\hat{H}\hat{I}}{\hat{H}+\hat{I}+\varepsilon}\,\rho(\Delta)$